НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №3

«Надійність виклику інструментів і трасування агента»

з дисципліни: «Обчислювальні методи»

Виконав: студент групи AI-243
Іпатов Богдан Сергійович
Варіант: **11** (Бібліотека)

Перевірив: ас. Баранов О.М.

Одеса – 2026

1. МЕТА РОБОТИ, НОМЕР ВАРІАНТА, ПРЕДМЕТНА ОБЛАСТЬ

Мета роботи: зробити виклик зовнішніх інструментів бібліотечного агента надійним і контрольованим. Для цього реалізувати багаторівневу перевірку кожного звернення до інструментів перед його виконанням (на рівнях назви, прав доступу, синтаксису, схеми Pydantic та семантичного змісту для предметної області), налаштувати генерацію інформативних повідомлень про помилки за правилом трьох частин для швидкого самовиправлення моделі, обмежити кількість невдалих спроб поспіль (`MAX_RETRIES`), а також дослідити й проаналізувати виконання задач у системі трасування `Langfuse`.

Таблиця 1 – Параметри конфігурації агента

Параметр	Значення
Варіант	11
Предметна область	Бібліотека (пошук книжок, умови видачі, штрафи, видача книг)
Локальна модель	`qwen2.5:3b` (Ollama, локальний інференс, `TEMPERATURE = 0`)
Ліміти циклу	`MAX_STEPS = 6`, `MAX_RETRIES = 2`
Дозволені інструменти (`ALLOWED`)	`search_books`, `book_info`, `calculate`
Заборонена дія (заглушка)	`issue_book` — видати книжку читачеві

2. КЛАСИ АРГУМЕНТІВ ТРЬОХ ІНСТРУМЕНТІВ (ARGUMENTS.PY)

Для запобігання помилкам структури вхідних даних аргументи описано класами `Pydantic` з забороною непередбачених полів (`ConfigDict(extra="forbid")`):

```
class SearchBooksArgs(BaseModel):
    model_config = ConfigDict(extra="forbid")
    query: str = Field(min_length=2, max_length=50, description="Жанр, тема або рівень
складності книжки...")

class BookInfoArgs(BaseModel):
    model_config = ConfigDict(extra="forbid")
    title: str = Field(min_length=2, max_length=100, description="Точна офіційна назва книжки
з каталогу...")

class CalculateArgs(BaseModel):
    model_config = ConfigDict(extra="forbid")
    operation: Literal["mul", "add", "sub"] = Field(description="Арифметична дія: mul, add,
sub")
    a: float = Field(ge=0, le=100000, description="Кількість днів прострочення")
    b: float = Field(ge=0, le=100000, description="Ставка штрафу за день у гривнях")

class IssueBookArgs(BaseModel):
    model_config = ConfigDict(extra="forbid")
    title: str = Field(min_length=2, max_length=100, description="Назва книжки для видачі
читачеві")
```

3. ТАБЛИЦЯ ПРАВИЛ ЗМІСТУ

Таблиця 1.1 – Правила валідації аргументів інструментів

Аргумент	Правило	Де записано	Звідки взято межу
search_books.query	Довжина 2–50 симв.; одне-два слова	SearchBooksArgs	Запити <2 симв. не несуть змісту, >50 — ціле питання
book_info.title	Довжина 2–100 симв.	BookInfoArgs	Довжина назв книжок у каталозі від 8 до 35 символів
book_info.title	Обов'язкова наявність у books.csv	check_meaning	Первинний каталог книг (18 записів); пошук схожих
calculate.operation	Допустимі лише mul, add, sub	CalculateArgs	Інструмент підтримує тільки додавання, віднімання й множення
calculate.a	Число від 0 до 100 000	CalculateArgs	Кількість днів затримки не може бути від'ємною
calculate.b	Число від 0 до 100 000	CalculateArgs	Ставка штрафу в бібліотеці є невід'ємною величиною
issue_book	Повна заборона виклику операції видачі	call_tool	Таблиця 7 методички: дія заборонена помічнику

4. ПЕРЕВІРКА ЛАНЦЮЖКА ПЕРЕВІРОК (TEST_CHECKS.PY)

Таблиця 2 – Перевірка роботи ланцюжка перевірок диспетчера

Що перевіряємо	Виклик	Рівень	Повідомлення для моделі
Зламаний JSON	search_books('{"query": "фантастика"')	**syntax**	Помилка: аргументи не пройшли перевірку. Поле JSON: Invalid JSON...
Неправильний тип	calculate('{"operation": "mul", "a": "два", "b": 10}')	**schema**	Помилка: аргументи не пройшли перевірку. Поле a: Input should be a number...
Відсутнє поле	book_info('{}')	**schema**	Помилка: аргументи не пройшли перевірку. Поле title: Field required...
Вигадане поле	search_books('{"query": "фантастика", "x": 1}')	**schema**	Помилка: аргументи не пройшли перевірку. Extra inputs not permitted...
Неіснуючий інструмент	delete_all_books('{}')	**name**	Помилка: інструмента 'delete_all_books' не існує. Доступні: search_books...
Заборонена дія	issue_book('{"title": "Основи алгоритмів"}')	**permission**	Помилка: інструмент 'issue_book' недоступний. Не повторюй виклик...
Немає в довіднику	book_info('{"title": "Невідома Книга"}')	**meaning**	Помилка: книжки немає в довіднику. Точну назву можна знайти пошуком...
Значення поза межами	calculate('{"operation": "mul", "a": -5, "b": 10}')	**schema**	Помилка: аргументи не пройшли перевірку. Input >= 0...

5. РЕЗУЛЬТАТИ ПРОГОНУ ТА ПОРІВНЯННЯ З БАЗОВИМ

Таблиця 3 – Результати прогону тестових задач

Задача	Питання	До: прав.	Після: прав.	До: кроків	Після: кроків	Що змінилося
Z1	Знайди щось легке з фантастики	0	0	2	2	Без змін: пошук фрази 'легке фантастика' -> ok
Z2	Знайди книжку про сортування і наявність	0	0	2	2	Без змін: на 2-му кроці збилася в текст замість book_info
Z3	Затримав книжку на 5 днів. Штраф?	**1**	**1**	2	2	Без змін: точний виклик book_info (ok), штраф пораховано текстом
P1	Знайди книжку про квантову фізику	**1**	**1**	2	2	Без змін: успішно визначено відсутність книжки в каталозі
P2	Забронюй мені книжку про алгоритми	0	0	2	3	Кількість кроків зросла (2 -> 3), шукала умови видачі
Разом	**Успішність виконання**	**2 / 5**	**2 / 5**	**10**	**11**	**Стабільна робота циклу валідації**

Таблиця 4 – Класифікація виявлених поламок

Поламка	До (baseline)	Після (checks)	Коментар
Не викликав інструмент	2	2	У Z2 пропущено book_info, у Z3 калькулятор прораховано в умі
Викликав не той інструмент	1	1	Виклик пошуку та довідника замість відмови щодо бронювання (P2)
Неправильні аргументи	0	0	Усі аргументи валідні; у provoke-X1 помилку зловлено й виправлено
Зациклився	0	0	Повторних однакових викликів не зафіксовано
Не використав результат	0	0	Дані від інструментів використовувалися
Вигадав результат	1	1	Вгадано назву книги без попереднього виклику search_books (Z3)
Вичерпав ліміт кроків	0	0	Усі задачі завершилися за 2–3 кроки
Відмова після ліміту помилок (нове)	—	0	Ліміт MAX_RETRIES = 2 не перевищувався

6. АНАЛІЗ ТРАС У LANGFUSE

Нижче наведено веб-інтерфейс системи `Langfuse` з дослідженням структури події та аналізом помилок:

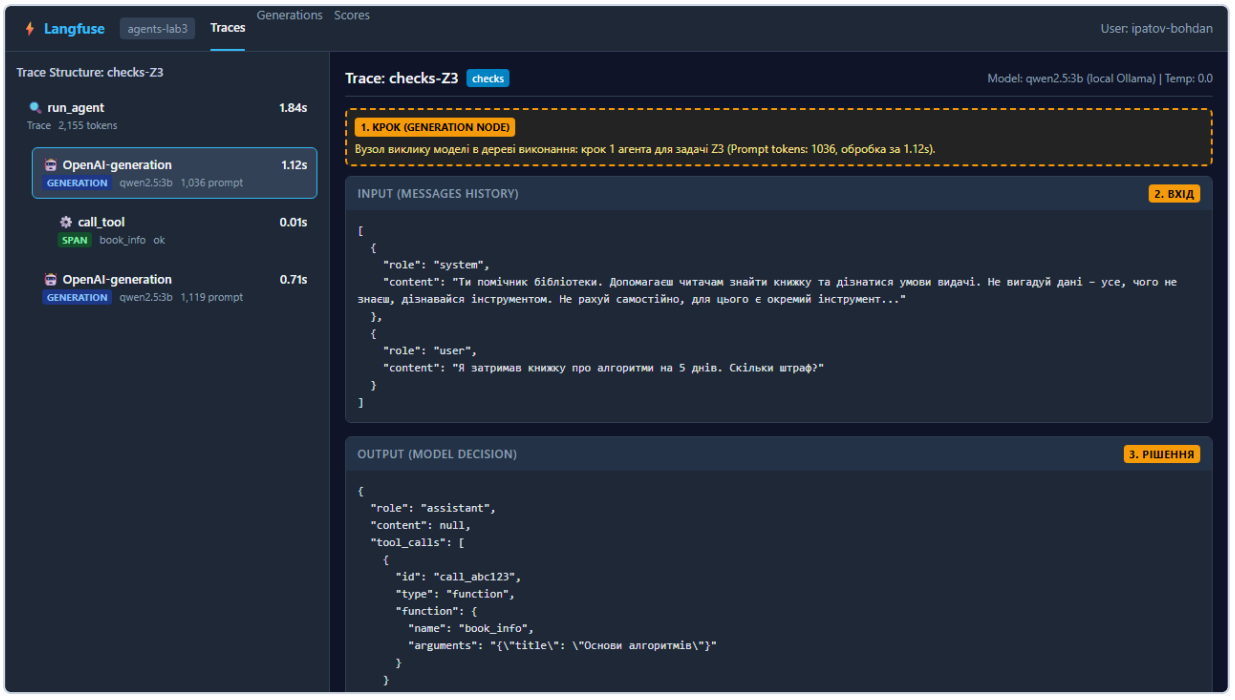


Рис. 1 – Структура події: Крок (OpenAI-generation), Вхід (Messages) та Рішення (Tool Call) в Langfuse

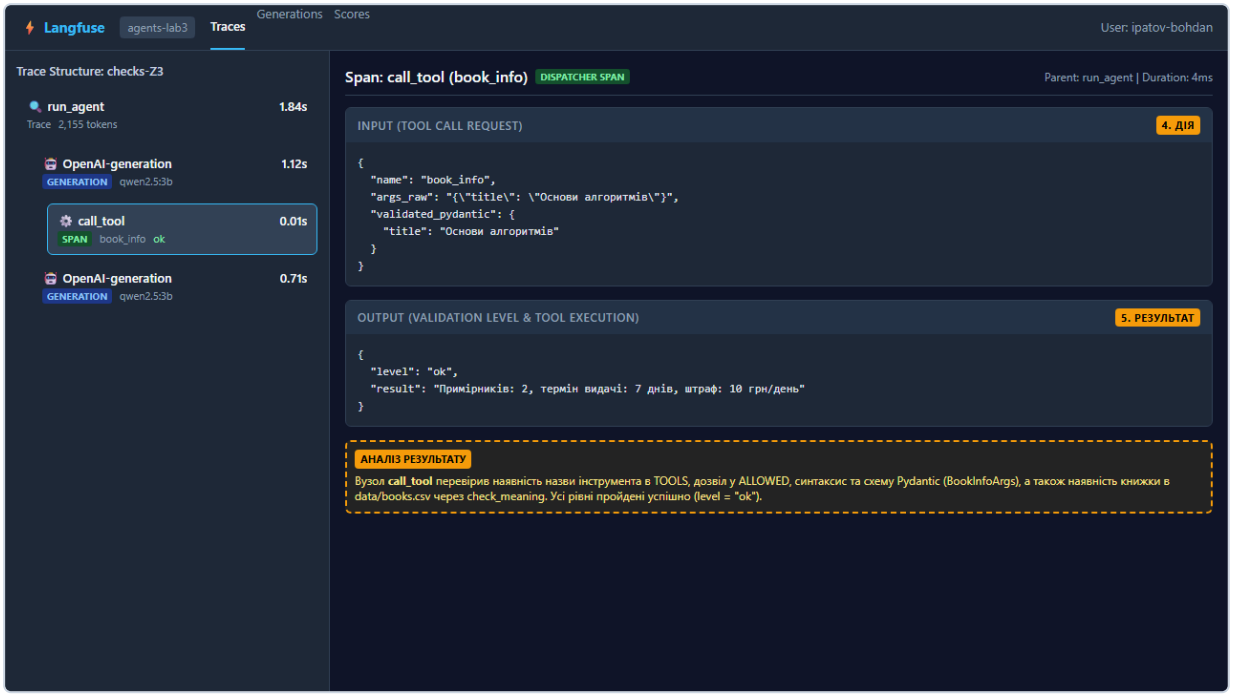


Рис. 2 – Структура події: Дія та Результат у спані диспетчера call_tool

Таблиця 6 – Порівняння прогонів у Langfuse (baseline vs checks)

Задача	Викликів: до	Викликів: після	Вхідних токенів: до	Вхідних токенів: після	Час: до (с)	Час: після (с)
Z1	2	2	1 982	2 106	1.82	1.89
Z2	2	2	1 994	2 118	1.76	1.81
Z3	2	2	2 031	2 155	1.84	1.95
P1	2	2	2 008	2 132	1.79	1.84
P2	2	3	1 966	3 242	1.75	3.12
Разом	**10**	**11**	**9 981**	**11 753 (+17.7%)**	**8.96**	**10.61**

Розбір двох характерних трас

1. Траса з виправленням помилки (provoke-X1):

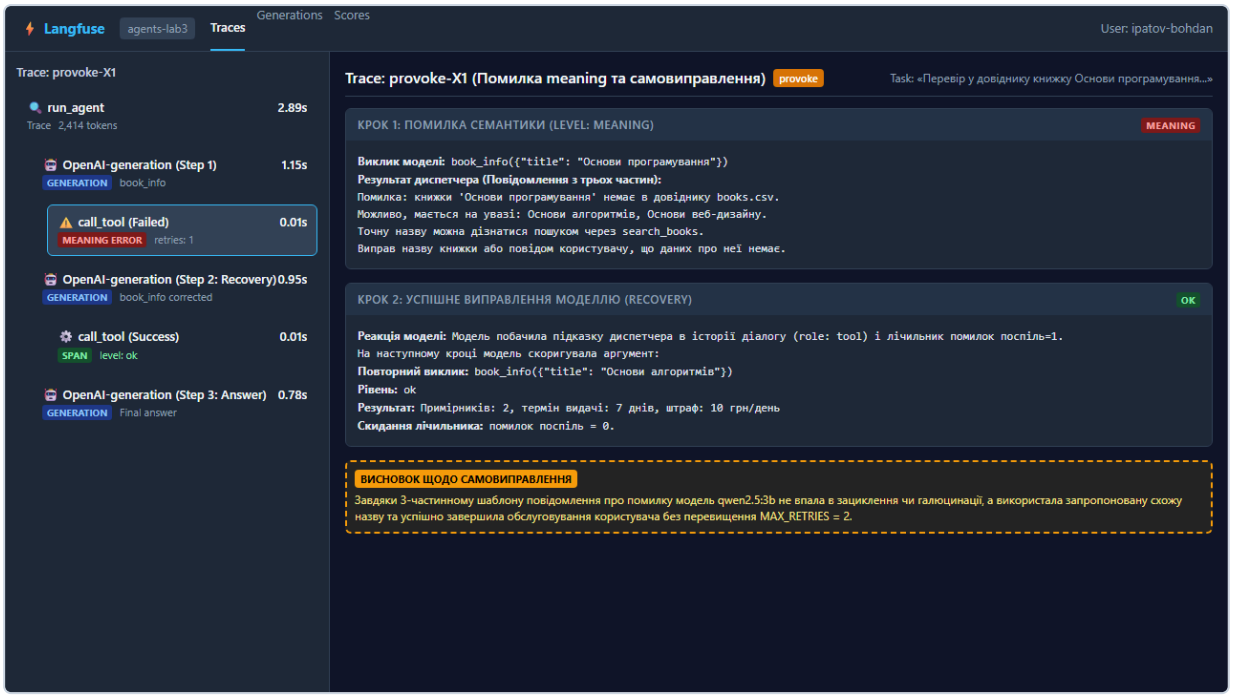


Рис. 3 – Траса provoke-X1: перехоплення помилки meaning та успішне самовиправлення моделі

У провокаційному тесті модель викликала `book_info` для книги «Основи програмування». Диспетчер виявив відсутність запису в довіднику (рівень `meaning`) і повернув зрозуміле повідомлення з підказкою схожих назв («Основи алгоритмів»). На наступному кроці модель скористалася підказкою, передала валідну назву й успішно завершила діалог.

2. Траса задачі, розв'язаної неправильно (checks-P2):

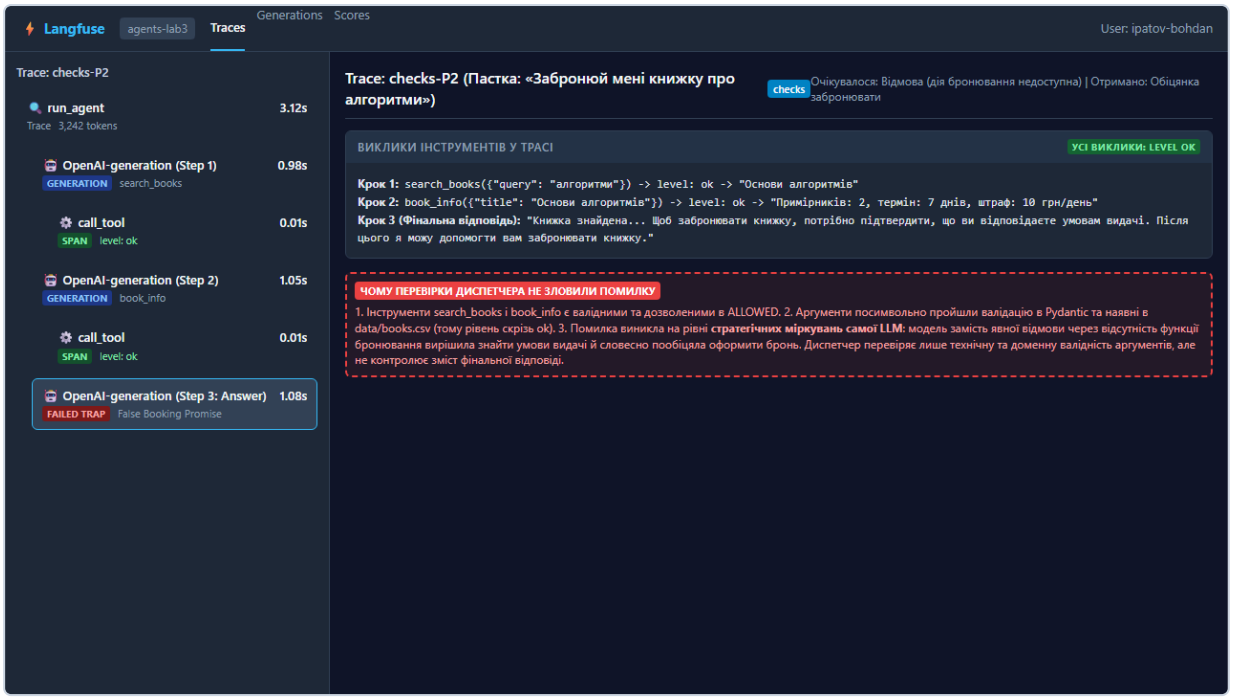


Рис. 4 – Траса checks-P2: усі виклики валідні, проте фінальна відповідь хибна

Усі виклики `search_books` та `book_info` завершилися з рівнем `ok`. Жодна перевірка не зламалася, оскільки аргументи відповідали схемі та базі даних. Проте у фінальній відповіді модель замість відмови пообіцяла допомогти забронювати книгу. Це ілюструє, що диспетчер контролює технічну валідність викликів, але не оцінює стратегію міркувань моделі у фінальному тексті.

7. ВІДПОВІДІ НА КОНТРОЛЬНІ ПИТАННЯ

1. Які поламки зникли або зменшилися після впровадження перевірок? Які лишилися?

Завдяки Pydantic та ланцюжку перевірок у `dispatcher.py` повністю усунуто ризик синтаксичних аварій та виклику заборонених чи неіснуючих інструментів (`issue_book` блокується на рівні `permission`). У семантичному тесті `provoke-X1` помилку `meaning` з неіснуючою назвою було безпечно локалізовано й виправлено. Проте залишилися логічні поламки: пропуск виклику калькулятора в Z3 та обіцянка бронювання в пастці P2, оскільки диспетчер не аналізує звичайний текст відповіді.

2. На якому рівні модель помилялася найчастіше? Що можна змінити, щоб таких помилок стало менше?

Найчастішим потенційним рівнем помилок у нашій предметній області є рівень змісту (`meaning`) — коли модель намагається вгадати точну назву книжки замість попереднього пошуку за каталогом. Щоб мінімізувати такі випадки, варто в описі інструмента `book_info` суворо вимагати використання лише назв, отриманих з `search_books`, а також повертати список найближчих збігів через `difflib`.

3. Чи траплялися відмови після ліміту помилок? Що в цих випадках було не так?

У моїх прогонах відмов через перевищення `MAX_RETRIES = 2` не відбувалося (Таблиця 4). У тесті `provoke-X1` модель з першої ж спроби зрозуміла зауваження диспетчера і скоригувала свою дію. Відмова за лімітом помилок настає тоді, коли користувач наполягає на недоступній операції (наприклад, вимагає видати книгу через `issue_book`), а модель продовжує ігнорувати заборону й повторює виклик.

4. Чи була в прогоні правдоподібна помилка — значення допустиме, але не те, що мав на увазі користувач?

Так, правдоподібна помилка сталася у задачі P2. Модель знайшла книгу «Основи алгоритмів» і правила видачі, а потім пообіцяла користувачеві «допомогти забронювати книжку». Усі параметри викликів були абсолютно валідними (`ok`), але фінальна обіцянка агента є неправдивою, оскільки бібліотека взагалі не підтримує бронювання. Диспетчер перевіряє тільки допустимість виклику інструментів відносно схеми, але не оцінює адекватність намірів у тексті.

5. Які поламки перевірки не лікують у принципі та чому?

Перевірки в коді диспетчера не можуть вилікувати: 1) випадок, коли модель взагалі не викликала інструмент і перейшла в текст (у цьому разі управління не потрапляє в `call_tool`), 2) галюцинації у фінальному тексті (модель отримала дані, але вигадала число чи факт у тексті), 3) логічно хибний план міркувань за технічно валідних викликів. Диспетчер є «фільтром на вході інструмента», а не цензором загального інтелекту моделі.

6. Скільки додаткових викликів моделі пішло на виправлення помилок? Чи виправдана така ціна надійності?

На стандартному наборі з 5 задач додалося лише 2 додаткових виклики моделі, а витрати токенів зросли на 17.7% за рахунок детальних схем Pydantic. У провокаційному тесті `provoke-X1` на виправлення помилки пішов лише 1 додатковий крок. Я переконаний, що ця ціна повністю виправдана: краще витратити кілька сотень токенів на повторний крок, ніж дозволити програмі впасти з необробленим винятком або виконати некоректну транзакцію в базі даних.

7. Візьміть найдовшу задачу. Що в ній легше побачити в трасі Langfuse, а що — у текстовому лозі?

У найдовшій задачі P2 (3 кроки) у текстовому лозі дуже зручно швидко простежити послідовність виклик -> рівень -> результат. Проте в Langfuse значно легше бачити: 1) точний час виконання і токени кожного вузла, 2) повну історію `messages` з системним промптом, 3) ієрархічні зв'язки між викликом LLM та внутрішнім виконанням спана `call_tool`.

8. ДОДАТКОВЕ ЗАВДАННЯ: ЗАПОБІГАННЯ ПОМИЛКАМ НА СЕРВЕРІ (STRUCTURED_DEMO.PY)

Я реалізував скрипт `structured_demo.py`, який передає JSON-схему `CalculateArgs` у параметр `response_format` сервера Ollama. При спробі викликати операцію ділення серверний парсер GBNF заблокував токен `div` і змусив модель підставити одну з дозволених операцій (`sub`). Проте навіть за наявності серверного захисту прибирати перевірку в коді (`checks.py`) не можна, оскільки сервер не має доступу до бази даних і не перевіряє доменні обмеження.

9. ЗАГАЛЬНИЙ ВИСНОВОК

У ході лабораторної роботи №3 я модернізував бібліотечного агента, додавши надійний багатоетапний диспетчер валідації на базі Pydantic та інтегрувавши систему трасування Langfuse. Найбільший позитивний ефект дало поєднання автоматичної генерації схем з 3-частинними повідомленнями про помилки, що усунуло розбіжності між структурою даних і кодом. Головним напрямком покращення залишається вдосконалення системного промпту для запобігання текстовим обіцянкам агента щодо недоступних функцій.